

```
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <sstream>
#include <string>
#include <vector>
```

```
#include <opencv2/dnn.hpp>
#include <opencv2/imgcodecs.hpp>
#include <opencv2/imgproc.hpp>
```

```
#include "openvino/openvino.hpp"
```

```
// -----
// Parse the 'names' section from a YOLO metadata.yaml.
// Expected format:
// names:
//   0: person
//   1: bicycle
// -----
static std::map<int, std::string> load_class_names(const std::string& yaml_path) {
    std::map<int, std::string> names;
    std::ifstream f(yaml_path);
    if (!f.is_open())
        return names;

    bool in_names = false;
    std::string line;
    while (std::getline(f, line)) {
        if (line.find("names:") != std::string::npos) {
            in_names = true;
            continue;
        }
        if (!in_names)
            continue;

        // Stop at the next top-level key (no leading whitespace, not a digit line)
        if (!line.empty() && line[0] != ' ' && line[0] != '\t')
            break;

        size_t colon = line.find(':');
        if (colon == std::string::npos)
            continue;

        std::string key_str = line.substr(0, colon);
        key_str.erase(0, key_str.find_first_not_of(" \t"));
        if (key_str.empty() || !std::isdigit(static_cast<unsigned char>(key_str[0])))
            continue;

        int id = std::stoi(key_str);
        std::string val = line.substr(colon + 1);
        val.erase(0, val.find_first_not_of(" \t"));
        if (!val.empty() && val.back() == '\r')
            val.pop_back();
        names[id] = val;
    }
    return names;
}
```

```
// -----
// Minimal JSON helpers - no third-party library required.
// -----
static std::string json_str(const std::string& s) {
    return "\"" + s + "\"";
}
```

```
static std::string utc_timestamp() {
    auto now = std::chrono::system_clock::now();
    std::time_t t = std::chrono::system_clock::to_time_t(now);
    std::tm tm_utc{};
    gmtime_r(&t, &tm_utc);
    char buf[32];
    std::strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S+00:00", &tm_utc);
    return buf;
}
```

```
// -----
int main(int argc, char* argv[]) {
    try {
        if (argc != 4) {
            std::cout << "Usage: " << argv[0]
                << " <path_to_model> <path_to_image> <device_name>" << std::endl;
            std::cout << "Example: " << argv[0]
                << " models/yolo/yolo26n_openvino_model/yolo26n.xml data/images/person/person_detection.png GPU"
                << std::endl;
            return EXIT_FAILURE;
        }
    }
```

```
    const std::string model_path{argv[1]};
    const std::string image_path{argv[2]};
    const std::string device_name{argv[3]};
```

```
    constexpr float CONF_THRESHOLD = 0.5f;
    constexpr float NMS_IΟΥ_THRESHOLD = 0.45f;
    constexpr int INPUT_SIZE = 640;
```

```
    // Load class names from metadata.yaml alongside the model
    const std::string dir =
        model_path.substr(0, model_path.find_last_of("/") + 1);
    const std::map<int, std::string> class_names =
        load_class_names(dir + "metadata.yaml");
```

```
// ----- Step 1. Initialize OpenVINO Runtime Core -----
std::cout << "[ INFO ] Creating OpenVINO Runtime Core" << std::endl;
ov::Core core;
```

```
// ----- Step 2. Read a model -----
std::cout << "[ INFO ] Reading the model: " << model_path << std::endl;
std::shared_ptr<ov::Model> model = core.read_model(model_path);
```

```
    if (model->inputs().size() != 1) {
        std::cerr << "[ ERROR ] Sample supports only single input topologies"
            << std::endl;
        return EXIT_FAILURE;
    }
```

```
// ----- Step 3. Set up input -----
cv::Mat image = cv::imread(image_path);
if (image.empty()) {
    std::cerr << "[ ERROR ] Cannot read image: " << image_path << std::endl;
    return EXIT_FAILURE;
}
```

```
const int orig_h = image.rows;
const int orig_w = image.cols;
```

```
    // Resize to INPUT_SIZE x INPUT_SIZE - do NOT reshape the model,
    // because transformer attention layers have incompatible fixed internal shapes.
    cv::Mat resized;
    cv::resize(image, resized, cv::Size(INPUT_SIZE, INPUT_SIZE));
```

```
    // Convert BGR -> RGB, normalize to [0, 1] as float32
    cv::Mat rgb;
    cv::cvtColor(resized, rgb, cv::COLOR_BGR2RGB);
    rgb.convertTo(rgb, CV_32F, 1.0 / 255.0);
```

```
    // HWC -> NCHW: split per-channel and lay out contiguously
    std::vector<cv::Mat> ch(3);
    cv::split(rgb, ch);
    std::vector<float> input_data(1 * 3 * INPUT_SIZE * INPUT_SIZE);
    for (int c = 0; c < 3; ++c) {
        const float* src = reinterpret_cast<const float*>(ch[c].data);
        std::copy(src, src + INPUT_SIZE * INPUT_SIZE,
            input_data.data() + c * INPUT_SIZE * INPUT_SIZE);
    }
```

```
// ----- Step 4. Load model to device -----
std::cout << "[ INFO ] Loading the model to the plugin" << std::endl;
ov::CompiledModel compiled_model = core.compile_model(model, device_name);
```

```
// ----- Step 5. Inference synchronously -----
std::cout << "[ INFO ] Starting inference in synchronous mode" << std::endl;
ov::InferRequest infer_request = compiled_model.create_infer_request();

ov::Tensor input_tensor(
    ov::element::f32,
    {1, 3, static_cast<size_t>(INPUT_SIZE), static_cast<size_t>(INPUT_SIZE)},
    input_data.data());
infer_request.set_input_tensor(input_tensor);

auto t0 = std::chrono::high_resolution_clock::now();
infer_request.infer();
auto t1 = std::chrono::high_resolution_clock::now();
const double inference_ms =
    std::chrono::duration<double, std::milli>(t1 - t0).count();
```

```
// ----- Step 6. Process output -----
// YOLO exported model output: [1, 300, 6] - NMS already applied by the model
// Each row: [x1, y1, x2, y2, score, class_id] in INPUT_SIZE pixel space
const ov::Tensor output_tensor = infer_request.get_output_tensor();
const ov::Shape out_shape = output_tensor.get_shape(); // [1, 300, 6]
const size_t num_boxes = out_shape[1];
const size_t box_size = out_shape[2]; // 6
const float* raw = output_tensor.data<const float>();
```

```
const float scale_x = static_cast<float>(orig_w) / INPUT_SIZE;
const float scale_y = static_cast<float>(orig_h) / INPUT_SIZE;
```

```
    // Although NMS is built into the model, the threshold may be set too loosely,
    // resulting in highly overlapping boxes being retained simultaneously (common
    // with float16 anchor arithmetic). Apply a second NMS pass here.
```

```
struct Candidate {
    int class_id;
    std::string label;
    float confidence;
    cv::Rect bbox; // in original image coordinates
};
```

```
    // First pass: collect all candidates above confidence threshold
    std::vector<Candidate> candidates;
    std::vector<cv::Rect> boxes_vec;
    std::vector<float> scores_vec;
```

```
    for (size_t i = 0; i < num_boxes; ++i) {
        const float* det = raw + i * box_size;
        const float score = det[4];
        if (score < CONF_THRESHOLD)
            continue;
```

```
        const int xmin = static_cast<int>(det[0] * scale_x);
        const int ymin = static_cast<int>(det[1] * scale_y);
        const int xmax = static_cast<int>(det[2] * scale_x);
        const int ymax = static_cast<int>(det[3] * scale_y);
        const int cid = static_cast<int>(det[5]);
```

```
        auto it = class_names.find(cid);
        const std::string label =
            (it != class_names.end()) ? it->second : "class_" + std::to_string(cid);
```

```
        cv::Rect bbox(xmin, ymin, xmax - xmin, ymax - ymin);
        candidates.push_back({cid, label, score, bbox});
        boxes_vec.push_back(bbox);
        scores_vec.push_back(score);
    }
```

```
    // Second pass: apply NMS to suppress overlapping duplicates
    std::vector<int> keep_indices;
    if (!boxes_vec.empty())
        cv::dnn::NMSBoxes(boxes_vec, scores_vec,
            CONF_THRESHOLD, NMS_IΟΥ_THRESHOLD, keep_indices);
```

```
    // Draw bounding boxes on the original image and build JSON output
    std::ostringstream detections_json;
    for (size_t i = 0; i < keep_indices.size(); ++i) {
        const Candidate& c = candidates[keep_indices[i]];
        cv::rectangle(image, c.bbox, cv::Scalar(0, 255, 0), 2);
```

```
        if (i > 0)
            detections_json << ",\n";
        detections_json
            << "    {\n"
            << "        \"id\": " << i << ",\n"
            << "        \"class_id\": " << c.class_id << ",\n"
            << "        \"label\": " << json_str(c.label) << ",\n"
            << "        \"confidence\": " << std::fixed << std::setprecision(4)
            << c.confidence << ",\n"
            << "        \"bbox\": {\n"
            << "            \"xmin\": " << c.bbox.x << ",\n"
            << "            \"ymin\": " << c.bbox.y << ",\n"
            << "            \"xmax\": " << c.bbox.x + c.bbox.width << ",\n"
            << "            \"ymax\": " << c.bbox.y + c.bbox.height << ",\n"
            << "        }\n"
            << "    }";
    }
```

```
    std::ostringstream output_json;
    output_json
        << "{\n"
        << "    \"timestamp\": " << json_str(utc_timestamp()) << ",\n"
        << "    \"image\": " << json_str(image_path) << ",\n"
        << "    \"model\": " << json_str(model_path) << ",\n"
        << "    \"device\": " << json_str(device_name) << ",\n"
        << "    \"inference_ms\": " << std::fixed << std::setprecision(2)
        << inference_ms << ",\n"
        << "    \"num_detections\": " << keep_indices.size() << ",\n"
        << "    \"detections\": [\n" << detections_json.str() << "\n"
        << "    ]\n"
        << "};";
```

```
    std::cout << "[ INFO ] Detection results:\n" << output_json.str() << std::endl;
```

```
    cv::imwrite("out.bmp", image);
    if (std::ifstream("out.bmp").good())
        std::cout << "[ INFO ] Image out.bmp was created!" << std::endl;
    else
        std::cerr << "[ ERROR ] Image out.bmp was not created. Check your permissions."
            << std::endl;
```

```
    } catch (const std::exception& ex) {
        std::cerr << ex.what() << std::endl;
        return EXIT_FAILURE;
    }
```

```
    std::cout << "\nThis sample is an API example, for any performance measurements "
        "please use the dedicated benchmark_app tool"
        << std::endl;
    return EXIT_SUCCESS;
}
```

openvino/src/inference/include/openvino/runtime/core.hpp

```
namespace ov {  
/**  
 * @brief This class represents an OpenVINO runtime Core entity.  
 * @ingroup ov_runtime_cpp_api  
 * User applications can create several Core class instances. In that case the device plugins  
 * will still share underlying resources (such as OCL context) in per-device singleton.  
 */  
class OPENVINO_RUNTIME_API Core {  
    class Impl;  
    std::shared_ptr<Impl> _impl;  
  
public:  
    ///@{  
    /** @brief Constructs an OpenVINO Core instance with devices  
     * and their plugins description.  
     *  
     * There are two ways how to configure device plugins:  
     * 1. (default) Use XML configuration file in case of dynamic libraries build;  
     * 2. Use strictly defined configuration in case of static libraries build.  
     *  
     * @param xml_config_file Path to the .xml file with plugins to load from. If path contains only file name  
     * with extension, file will be searched in a folder with OpenVINO runtime shared library.  
     * If the XML configuration file is not specified, default OpenVINO Runtime plugins are loaded from:  
     * 1. (dynamic build) default "plugins.xml" file located in the same folder as OpenVINO runtime shared library;  
     * 2. (static build) statically defined configuration. In this case path to the .xml file is ignored.  
     */  
    explicit Core(const std::string& xml_config_file = {});  
  
    explicit Core(const std::filesystem::path& xml_config_file);  
};
```

openvino/src/inference/src/dev/core_impl.hpp

```
// Mutex is needed to prevent changes of dev mutexes map from different threads  
mutable std::mutex m_global_mutex;  
// Global mutex "" locks parallel access to m_plugin_registry and plugins  
// Plugin mutexes "plugin_name" lock access to code which changes configuration of particular plugin  
mutable std::unordered_map<std::string, std::mutex> m_dev_mutexes;  
  
std::mutex& get_mutex(const std::string& dev_name = "") const;  
void add_mutex(const std::string& dev_name);
```

openvino/src/inference/src/dev/core_impl.cpp

```
void ov::CoreImpl::add_mutex(const std::string& dev_name) {    // dev_name == ""  
    std::lock_guard<std::mutex> lock(m_global_mutex);  
    m_dev_mutexes[dev_name];  
    std::cerr << "[trace] add_mutex: \"" << dev_name << "\", total=" << m_dev_mutexes.size() << std::endl;  
}
```

```
// 1. 无参数版本 -> 取 key="" 的 mutex -> 全局锁  
get_mutex()          // 等价于 get_mutex("")
```

```
// 2. 有参数版本 -> 取具体设备的 mutex -> 设备级锁  
get_mutex("GPU")  
get_mutex("CPU")
```

openvino/src/core/include/openvino/core/version.hpp

```
/**  
 * @def OPENVINO_VERSION_MAJOR  
 * @brief Defines OpenVINO major version  
 *  
 * @def OPENVINO_VERSION_MINOR  
 * @brief Defines OpenVINO minor version  
 *  
 * @def OPENVINO_VERSION_PATCH  
 * @brief Defines OpenVINO patch version  
 */  
  
#define OPENVINO_VERSION_MAJOR 2026  
#define OPENVINO_VERSION_MINOR 2  
#define OPENVINO_VERSION_PATCH 0
```

ov::Core core;

openvino/src/inference/src/cpp/core.cpp

```
Core::Core(const std::string& xml_config_file) : Core(ov::util::make_path(xml_config_file)) {}
```

```
Core::Core(const std::filesystem::path& xml_config_file) : _impl(std::make_shared<Impl>()) {
```

```
class Core::Impl : public CoreImpl {  
public:  
    Impl() : ov::CoreImpl() {}  
};
```

openvino/src/inference/src/dev/core_impl.cpp

```
ov::CoreImpl::CoreImpl() {  
    add_mutex(""); // Register global mutex  
    m_executor_manager = ov::threading::executor_manager();  
    for (const auto& it : ov::get_available_opsets()) {  
        m_opset_names.insert(it.first);  
    }  
}
```

```
if (const auto xml_path = find_plugins_xml(xml_config_file); !xml_path.empty()) {  
    std::filesystem::path find_plugins_xml(const std::filesystem::path& xml_file) {  
        if (xml_file.has_parent_path()) {  
            // If file path contains file separator, return file path;  
            // Otherwise search it in OV folder with no restriction on file name and extension.  
            return xml_file;  
        }  
  
        // Default plugin xml file name, will search in OV folder.  
        const auto& xml_file_name = xml_file.empty() ? std::filesystem::path("plugins.xml") : xml_file;  
        const auto ov_library_path = ov::util::get_ov_lib_path();  
  
        // plugins xml can be found in either:  
        // 1. openvino-X.Y.Z relative to libopenvino.so folder  
        // 2. in folder with libopenvino.so  
        const auto ov_version_folder = [] {  
            std::ostringstream str;  
            // "openvino-2026.2.0"  
            str << "openvino-" << OPENVINO_VERSION_MAJOR << "." << OPENVINO_VERSION_MINOR << "." << OPENVINO_VERSION_PATCH;  
            return std::filesystem::path(str.str());  
        }();  
  
        if (auto xml_path = ov_library_path / ov_version_folder / xml_file_name; ov::util::file_exists(xml_path)) {  
            // register plugins from default openvino-openvino version>plugins.xml config  
            return xml_path;  
        } else if (xml_path = ov_library_path / xml_file_name; ov::util::file_exists(xml_path)) {  
            return xml_path;  
        } else {  
            return xml_file;    // xml_file_M_pathname == ""  
        }  
    }  
}
```

```
// If XML is default, load default plugins by absolute paths  
OV_CORE_CALL_STATEMENT(_impl->register_plugins_in_registry(xml_path, xml_config_file.empty()));  
}  
// Load plugins from the pre-compiled list  
OV_CORE_CALL_STATEMENT(_impl->register_compile_time_plugins());  
}
```

ov::Core core; 等价于 ov::Core core("");。空字符串先被 make_path 转成 std::filesystem::path{}，然后委派给 Core::Core(const std::filesystem::path&)

openvino/src/common/util/include/openvino/util/file_util.hpp

```
/**  
 * @brief Creates std::filesystem::path provided by source.  
 *  
 * The purpose of this function is to hide platform specific issue with path creation like on Windows create from  
 * literal std::string with unicode characters can lead to different path name than expected.  
 *  
 * @param source Source to create path. Supported types are same as for std::filesystem::path constructor.  
 * @return std::filesystem::path object.  
 */  
template <class Source>  
std::filesystem::path make_path(Source&& source) {  
    if constexpr (std::is_same_v<std::add_const_t<std::remove_pointer_t<decay_t<Source>>>, const wchar_t*>) {  
        return {std::wstring(std::forward<Source>(source))};  
    } else if constexpr (std::is_same_v<std::filesystem::path::string_type, std::wstring> &&  
        std::disjunction_v<std::is_same<std::decay_t<Source>, std::string>,  
        std::is_same<std::decay_t<Source>, const char*>,  
        std::is_same<std::decay_t<Source>, char*>,  
        std::is_same<std::decay_t<Source>, std::string_view>>) {  
        return {string_to_wstring(std::forward<Source>(source))};  
    } else {  
        return {std::forward<Source>(source)};    // source=""  
    }  
}
```

openvino/src/core/src/opsets/opset.cpp

```
const std::map<std::string, std::function<const ov::OpSet&()>>& ov::get_available_opsets() {  
    #define _OPENVINO_REG_OPSET(OPSET) \  
    { #OPSET, ov::get_##OPSET }  
    const static std::map<std::string, std::function<const ov::OpSet&()>> opset_map = {_OPENVINO_REG_OPSET(opset1),  
        _OPENVINO_REG_OPSET(opset2),  
        _OPENVINO_REG_OPSET(opset3),  
        _OPENVINO_REG_OPSET(opset4),  
        _OPENVINO_REG_OPSET(opset5),  
        _OPENVINO_REG_OPSET(opset6),  
        _OPENVINO_REG_OPSET(opset7),  
        _OPENVINO_REG_OPSET(opset8),  
        _OPENVINO_REG_OPSET(opset9),  
        _OPENVINO_REG_OPSET(opset10),  
        _OPENVINO_REG_OPSET(opset11),  
        _OPENVINO_REG_OPSET(opset12),  
        _OPENVINO_REG_OPSET(opset13),  
        _OPENVINO_REG_OPSET(opset14),  
        _OPENVINO_REG_OPSET(opset15),  
        _OPENVINO_REG_OPSET(opset16),  
        _OPENVINO_REG_OPSET(opset17)};  
    #undef _OPENVINO_REG_OPSET  
    return opset_map;  
}
```

```
std::shared_ptr<ov::Model> model = core.read_model(model_path);
```

openvino/src/inference/src/cpp/core.cpp

```
std::shared_ptr<ov::Model> Core::read_model(const std::string& model_path,
                                           const std::string& bin_path,
                                           const AnyMap& properties) const {
    std::cerr << "[trace] " << __PRETTY_FUNCTION__ << " @ " << __FILE__ << ":" << __LINE__ << std::endl;
    OV_ITT_SCOPED_REGION_BASE(ov::itt::domains::Phases, "Read model");
    return read_model(ov::util::make_path(model_path), ov::util::make_path(bin_path), properties);
}
```

```
std::shared_ptr<ov::Model> Core::read_model(const std::filesystem::path& model_path,
                                           const std::filesystem::path& bin_path,
                                           const ov::AnyMap& properties) const {
    std::cerr << "[trace] " << __PRETTY_FUNCTION__ << " @ " << __FILE__ << ":" << __LINE__ << std::endl;
    OV_ITT_SCOPED_REGION_BASE(ov::itt::domains::Phases, "Read model");
    OV_CORE_CALL_STATEMENT(return _impl->read_model(model_path, bin_path, properties));
}
```

openvino/src/inference/src/dev/core_impl.cpp

```
std::shared_ptr<ov::Model> ov::CoreImpl::read_model(const std::filesystem::path& model_path,
                                                    const std::filesystem::path& bin_path,
                                                    const AnyMap& properties) const {
    OV_ITT_SCOPE(FIRST_INFERENCE, ov::itt::domains::ReadTime, "CoreImpl::read_model from file");
    auto local_core_config = m_core_config;
    local_core_config.set(properties, {});
    return ov::util::read_model(model_path, bin_path, get_extensions_copy(), local_core_config.get_enable_mmap());
}
```

openvino/src/inference/src/model_reader.cpp

```
std::shared_ptr<ov::Model> read_model(const std::filesystem::path& model_path,
                                       const std::filesystem::path& bin_path,
                                       const std::vector<ov::Extension::Ptr>& extensions,
                                       bool enable_mmap) {
    // Try to load with FrontEndManager
    ov::frontend::FrontEndManager manager;
    ov::frontend::FrontEnd::Ptr FE;
    ov::frontend::InputModel::Ptr inputModel;

    ov::AnyVector params{model_path};

    if (!bin_path.empty()) {
        params.emplace_back(bin_path);
    }
    params.emplace_back(enable_mmap);

    FE = manager.load_by_model(params);
    if (FE) {
        FE->add_extension(extensions);
        inputModel = FE->load(params);
    }

    if (inputModel) {
        auto model = FE->convert(inputModel);
        update_v10_model(model);
        return model;
    }

    std::string FEs;
    for (const auto& fe_name : manager.get_available_front_ends())
        FEs += fe_name + " ";
    OPENVINO_THROW("Unable to read the model: ",
                  model_path,
                  " Please check that model format: ",
                  model_path.extension(),
                  " is supported and the model is correct. Available frontends: ",
                  FEs);
}
```